SOCIOL 723: Social Statistics II

Week 12, Machine Learning: Regularization, Trees, and Neural Networks

Wenhao Jiang
Department of Sociology, Fall 2026

Duke
UNIVERSITY

Why ML

Four Reasons to Use Machine Learning

1. **Prediction is the goal.** Where are the poor? Wealth in Rwanda predicted from mobile-phone records (Blumenstock, Cadamuro, and On 2015).
2. **Measurement.** Turn text and images into variables: how populist a speech is (Bonikowski, Luo, and Stuhler 2022); how rich a neighbourhood is, read from its cars (Gebru et al. 2017).
3. **Ingredients of causal estimators.** Weighting needs each person's chance of treatment given X; regression adjustment, their expected outcome (Week 6). With hundreds of covariates, or text and images as X, ML predicts them, and double machine learning keeps their errors out of the causal estimate (Chernozhukov et al. 2018; Imai and Nakamura 2026).
4. **Who benefits?** A causal forest splits on X to find where the treatment effect $\tau(x)$ is larger or smaller (Wager and Athey 2018).

Reason 2: text, after neural networks; 3 and 4: next week. Common thread: every job predicts something from X: the outcome Y, the treatment D, or the effect $\tau(x)$.

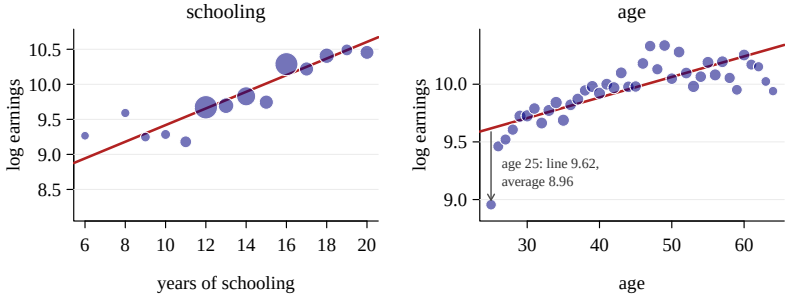
Three Kinds of Machine Learning ★

kind	the data	what is learned	examples
supervised	inputs X and a recorded outcome Y	a rule that predicts Y for new cases	log earnings: lasso, trees
unsupervised	inputs X only, no outcome	a few dimensions that summarize X	PCA; a status index
self-supervised	inputs X; hide a part and treat it as Y	a code that predicts the hidden part	word2vec, BERT; the hidden word

- **Supervised**: someone recorded the answer; judged by error on new cases.
- **Unsupervised**: no answer to check; judged by how much of X it keeps.
- **Self-supervised**: labels come free from the data, so it trains on billions of words.
- Today: supervised, then self-supervised (word2vec) after neural networks. PCA: next week.

Linear Regression Is Already Machine Learning ★

OLS learns a rule from data, $\hat{Y} = a + b\,x$, and applies it to anyone: a supervised learner with a shape fixed in advance. Blue dots: average log earnings at each value (size = number of people); red: the OLS rule (figs/w12_intro.R, A).



- Schooling: the line (slope 0.119) is close. Age: earnings rise, then flatten, and the line misses the young (arrow). A fixed shape cannot learn a pattern it does not allow.
- Next: rules whose shape is learned, and ways to stop them bending too much.

The Goal From Here: Predict People We Have Not Seen ★

From now on

Learn a rule from **training** data, then predict Y (log earnings, or any outcome) for **new** people. Every method is judged by its error on them, not by how well it fits the data it learned from.

Test R^2: compare the rule with the simplest one, "predict everyone with the average" $\bar{Y}$ of the training data:

$$R^2_{\text{test}} = 1 - \frac{\sum_{\text{new } i} \left(Y_i - \hat{Y}_i\right)^2}{\sum_{\text{new } i} \left(Y_i - \bar{Y}\right)^2}$$

- 1: perfect. 0: no better than the average. Negative: worse than the average.
- Why it can be negative: in the training data, OLS with an intercept can always match the average (set every slope to 0), so its $R^2 \geq 0$. On new people nothing stops a rule from doing worse than the average. Test R^2 has no floor.

Overfitting

Making OLS Flexible Backfires ★

The obvious fix: let OLS bend by adding squares and interactions. The 23 GSS columns become 243.

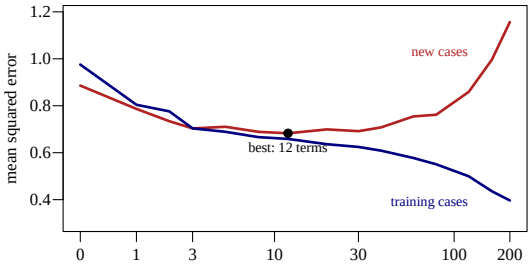
Fit on 300 training cases, predict 1,053 new people (figs/w12_gss.R):

	MSE, new people	test R^2
predict everyone with the average	0.886	0
OLS, 23 main effects	0.720	0.19
OLS, all 243 terms	11.302	−11.8

$1 - 11.302/0.886 = -11.8$. With 243 coefficients and 300 cases, OLS fits the training noise: **overfitting**. Machine learning = flexible *without* overfitting, plus a way to check.

Training Error Flatters; New Cases Tell the Truth ★

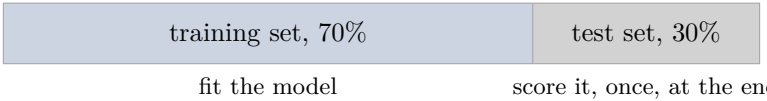
OLS on the first p of the 243 GSS columns, 600 training cases, 1,053 new cases (figs/w12_gss.R):



- Training error falls with every term added (0.98 with none, 0.40 with 200): a coefficient can always fit a little more noise.
- Error on new cases falls, then rises: 0.89 with none, 0.68 with 12, 1.16 with 200.
- So we cannot judge or tune a method on the data it was fitted to.

Measuring Error Honestly: Hold Out a Test Set ★

Training error is optimistic (previous slides). To measure error on *new* cases, keep some cases new: hide them from the fit.



1. Split the sample *at random*, before looking at anything.
2. Fit only on the training set.
3. Predict the test cases and compute the test MSE: an honest estimate of error on new cases, because the model never saw them.

Every GSS comparison in this deck uses one such split: 2,456 training cases, 1,053 test cases (the previous slide used 600 of the training cases).

Regularization

Why Regularization Is Machine Learning ★

- **The problem.** With 243 terms and 300 cases, OLS uses every coefficient to chase the training cases, noise included, and predicts new people worse than the average.
- **The idea.** Keep the linear regression, but make every coefficient pay for being nonzero, and more for being large. A predictor gets a sizeable coefficient only if it improves the fit by enough to cover the charge; weak ones are pulled toward zero, or set to zero.
- **The trade.** The pulled-in coefficients are a little off on average, but they barely move from sample to sample, so predictions for new people improve.
- **What makes it machine learning.** How much to charge is not chosen by the researcher or by theory. It is a tuning parameter chosen by the data: we try many charges and keep the one that predicts held-out cases best.
- The lasso and ridge are two ways to set the charge (next slides).

Why Would p Be Large?

- **Constructed features.** Flexible relationships need squares and interactions, and they multiply fast.
  - Civil war onset: the literature offers 88 country-level predictors (Hegre and Sambanis 2006). All pairwise interactions add 3,828 terms; three-way interactions add 109,736, far more than the 7,140 country-years, with only 115 onsets to learn from (Muchlinski et al. 2016).
  - The GSS: 23 main-effect columns become 243 with pairs and squares.
- **Many available features.** Text, images, and administrative records give thousands of columns. Most are noise; some proxy traits we want to model.
- When terms outnumber cases, or come close, the data are **high-dimensional**: OLS cannot be fitted, or fits the noise.

Lasso ★

Least **A**bsolute **S**hrinkage and **S**election **O**perator: $\hat{\beta}$ solves a *penalized* least squares problem

$$\hat{\beta} = \arg\min_{b \in \mathbb{R}^p} \sum_{i=1}^{n} \left(Y_i - \sum_j X_{ij} b_j\right)^2 + \lambda \sum_{j=1}^{p} |b_j|$$

- Fit in the sample, plus λ times the total absolute size of the coefficients.
- $\lambda = 0$ is OLS. As λ grows, more coefficients are set *exactly* to zero: lasso selects variables and estimates them at once.
- **Standardize** the regressors first (software does), so the penalty does not depend on whether income is in dollars or thousands.

GSS, 243 terms, 2,456 training cases (figs/w12_690s.R, C):

λ (glmnet scale)	0.35	0.024	0.0014
terms kept	0	42	179

Lasso: Fixing All But One Coefficient ★

The lasso minimizes, over $b_1, \ldots, b_p$,

$$L(b) = \sum_{i=1}^{n} \Big(Y_i - \sum_{k=1}^{p} X_{ik} b_k \Big)^2 + \lambda \sum_{k=1}^{p} |b_k|,$$

with regressors standardized: mean 0, variance 1, so $\sum_i X_{ij}^2 = n$.

One coefficient at a time. Fix every b_k with $k \neq j$ at its current value (the algorithm starts from all zeros). Let $R_i = Y_i - \sum_{k \neq j} X_{ik} b_k$, the part of Y_i the other regressors leave unexplained. As a function of b_j alone,

$$L(b_j) = \sum_{i=1}^{n} \big(R_i - X_{ij} b_j \big)^2 + \lambda \, |b_j| + \text{terms without } b_j.$$

Solve for b_j, update it, move to the next j using the updated values, and cycle until nothing changes (*coordinate descent*, `glmnet`). The loss is convex, so the starting values do not change the answer.

Lasso: the First-Order Condition ★

For $b_j \neq 0$, $|b_j|$ has derivative $\text{sign}(b_j)$ ($+1$ or -1). Set $\partial L/\partial b_j = 0$:

$$-2\sum_i X_{ij}(R_i - X_{ij}b_j) + \lambda\,\text{sign}(b_j) = 0 \qquad \text{chain rule}$$

$$-2\sum_i X_{ij}R_i + 2b_j\sum_i X_{ij}^2 + \lambda\,\text{sign}(b_j) = 0 \qquad \text{multiply out}$$

$$-2n\,\hat{b}_j + 2n\,b_j + \lambda\,\text{sign}(b_j) = 0 \qquad \textstyle\sum_i X_{ij}^2 = n;\ \hat{b}_j = \frac{1}{n}\sum_i X_{ij}R_i$$

$$b_j = \hat{b}_j - \frac{\lambda}{2n}\,\text{sign}(b_j) \qquad \text{solve for } b_j$$

- $\hat{b}_j$ is the least-squares answer: set $\lambda = 0$ and the condition gives $b_j = \hat{b}_j$.
- The lasso moves it toward zero by $\lambda/(2n)$: down if $b_j > 0$, up if $b_j < 0$.
- The formula assumes $b_j \neq 0$. Next: when is $b_j = 0$?

The Kink Decides: When Is b_j Zero?

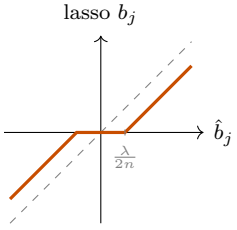
At $b_j = 0$, $|b_j|$ has a kink: slope -1 to the left, $+1$ to the right, so any value in $[-1, 1]$ is a valid slope there (the *subgradient*). Collect the three cases:

$$b_j > 0: \quad b_j = \hat{b}_j - \tfrac{\lambda}{2n}, \text{ which is positive only if } \hat{b}_j > \tfrac{\lambda}{2n}$$
$$b_j < 0: \quad b_j = \hat{b}_j + \tfrac{\lambda}{2n}, \text{ which is negative only if } \hat{b}_j < -\tfrac{\lambda}{2n}$$
$$b_j = 0: \quad 0 \in -2n\hat{b}_j + \lambda\,[-1, 1] \iff |\hat{b}_j| \le \tfrac{\lambda}{2n}$$

In one line (*soft thresholding*):

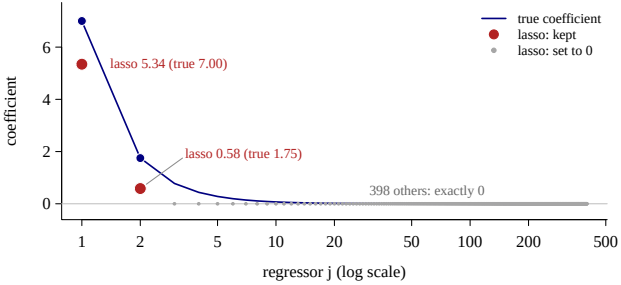
$$b_j = \operatorname{sign}(\hat{b}_j)\left(|\hat{b}_j| - \tfrac{\lambda}{2n}\right)_+$$

Move the least-squares answer toward zero by $\lambda/(2n)$; if that would cross zero, stop at zero. A weak regressor is dropped.



The Lasso When Data Are High-Dimensional ★

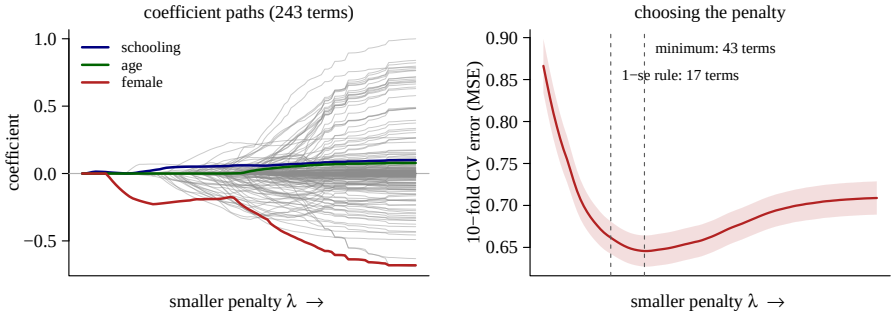
400 regressors, only 100 cases: OLS has no unique answer. True coefficients $7/j^2$: two matter (7 and 1.75); the other 398 are tiny (figs/w12_sparsity.R). Lasso with $\lambda = 0.4$:



- It keeps exactly the two that matter and sets the other 398 to 0, as the kink predicts.
- Shrinkage bias: the two come out at 5.34 and 0.58, pulled toward 0 from 7 and 1.75.
- A smaller λ (the CV minimum, 0.23) also lets in a few regressors that correlate with Y by chance, each with a tiny coefficient.

The Lasso Path on the GSS

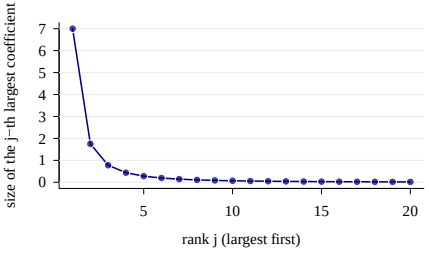
Lasso on the 243 GSS terms, 2,456 training cases, for 100 values of λ (figs/w12_gss.R):



- **Left**: one line per coefficient as the penalty falls. At a huge λ all are 0; then they enter one by one. Female enters first (a dummy: its coefficient is the whole gap).
- **Right**: CV error for each λ (± 1 s.e.): too large underfits, too small overfits. Minimum: 43 terms. The **1-se rule** takes the largest λ within one s.e. of the minimum: 17 terms, a simpler model that predicts about as well. How CV works: later.

Approximate Sparsity ★

- Lasso works best when the true coefficients are approximately sparse:
  - a small group of regressors with relatively large coefficients suffices to approximate the regression $\sum_j \beta_j X_{ij}$;
  - the rest have small coefficients and add little.
- A classic example: $\beta_j \propto 1/j^2$, $j = 1, \ldots, p$.



$\beta_j = 7/j^2$: 7 at $j = 1$, 1.75 at 2, 0.28 at 5, 0.07 at 10; beyond that they barely matter.

Ridge Regression for Dense Coefficients ★

- Lasso does best when a few coefficients are large; ridge when many are small:

$$\hat{\beta}^{\text{ridge}} = \underset{b \in \mathbb{R}^p}{\arg\min} \sum_{i=1}^{n} \left(Y_i - \sum_j X_{ij} b_j\right)^2 + \lambda \sum_{j=1}^{p} b_j^2$$

- The penalty is the squared (ℓ_2) size instead of the absolute (ℓ_1) size.
- Ridge penalizes *large* coefficients much more aggressively and *small* ones much less. The marginal cost of moving b_j:

	$b_j = 0.1$	$b_j = 1$	$b_j = 2$
lasso, $\partial(\lambda\lvert b_j\rvert)/\partial b_j = \lambda$	λ	λ	λ
ridge, $\partial(\lambda b_j^2)/\partial b_j = 2\lambda b_j$	0.2λ	2λ	4λ

- Near zero ridge's cost vanishes, so no coefficient is ever pushed exactly to 0: ridge keeps every regressor, each a little, which suits a dense truth.

Ridge, One Coefficient at a Time

Same steps with λb_j^2 in place of $\lambda|b_j|$. No kink, so one first-order condition:

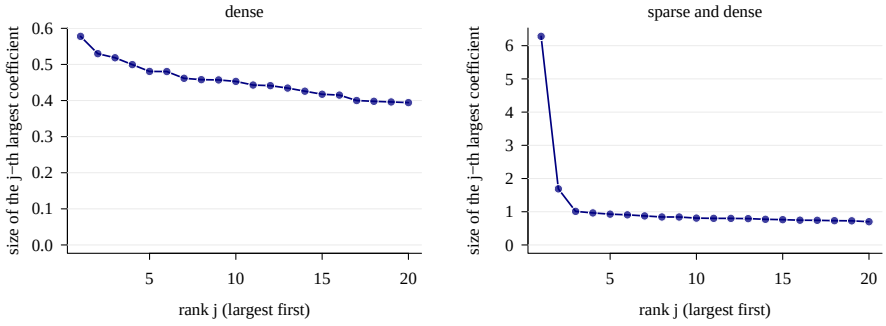
$$-2\sum_i X_{ij}(R_i - X_{ij}b_j) + 2\lambda b_j = -2n\hat{b}_j + 2nb_j + 2\lambda b_j = 0 \;\Rightarrow\; b_j = \hat{b}_j \cdot \frac{n}{n+\lambda}$$

Example: $n = 100$, ridge $\lambda = 25$ (factor $100/125 = 0.8$), lasso $\lambda = 40$ (threshold 0.20) (figs/w12_690s.R, B):

	OLS	ridge	lasso
strong regressor	0.30	$0.30 \times 0.8 = 0.24$	0.10
weak regressor	0.15	$0.15 \times 0.8 = 0.12$	0

- Ridge *multiplies* every coefficient by $n/(n+\lambda)$; lasso *subtracts* $\lambda/(2n)$ and stops at 0. Ridge never reaches zero, so it never selects.
- $\lambda = 0$ is OLS; $\lambda \to \infty$ sends every slope to 0.

Where Ridge Wins: Many Small Effects



- **Dense** (left): every coefficient small, none dominant. Ridge does best: it keeps every regressor and shrinks each a little, where the lasso would drop real small effects.
- Ridge does *not* select: all p regressors stay in, so it does not make the model smaller.

Elastic Net for Sparse *or* Dense Coefficients ★

- Ridge and lasso can be combined to do well in either *sparse* or *dense* settings. The elastic net, with tuned penalty levels:

$$\hat{\beta}^{\text{EN}} = \underset{b \in \mathbb{R}^p}{\arg\min} \sum_{i=1}^{n} \left(Y_i - \sum_j X_{ij} b_j\right)^2 + \lambda_1 \sum_{j=1}^{p} b_j^2 + \lambda_2 \sum_{j=1}^{p} |b_j|$$

- Choosing λ_1 and λ_2 gives more flexibility for a good prediction rule than ridge or lasso alone.
- It still selects variables unless we shut the lasso penalty off with $\lambda_2 = 0$.
- It handles groups of correlated predictors better than pure lasso: the ridge part spreads weight across the group instead of picking one.
- In `glmnet`: one λ and a mixing weight α, $\lambda\big(\alpha \sum |b_j| + \frac{1-\alpha}{2} \sum b_j^2\big)$; $\alpha = 1$ is lasso, $\alpha = 0$ ridge.

In the Literature: Poverty From Phone Records

The question. Blumenstock, Cadamuro, and On (2015): Rwanda has few reliable, recent measures of who is poor and where. Can mobile phone records stand in for a household survey?

The data. Phone surveys of 856 randomly sampled subscribers (assets, housing) give a wealth index; each respondent's call records give *several thousand* metrics: call volume, timing, contacts' network, mobility.

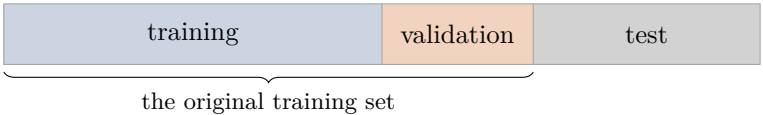
The method. $p \gg n$: thousands of candidate regressors for 856 people. Exactly our setting. Elastic net drops the irrelevant metrics; λ chosen by cross-validation.

The results.

- Cross-validated correlation between predicted and surveyed wealth: $r = 0.68$.
- Averaged over Rwanda's 30 districts, the predictions match the government's in-person DHS survey of 12,792 households: $r = 0.92$.
- Tree ensembles gave similar results: the gain came from the regularized search over many features, not from one special method.

Choosing a Tuning Parameter Needs a Third Piece ★

Every method today has a tuning parameter: the penalty λ, tree depth, network size. Three jobs, three pieces of data:

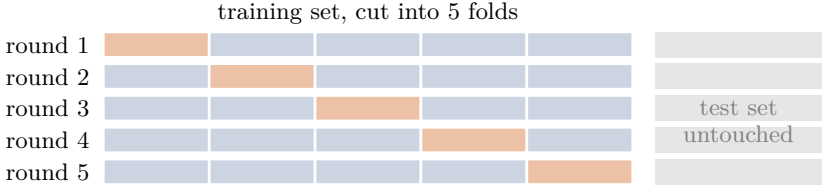


job	data	what goes wrong if you use the wrong piece
fit $\hat{f}$ for a given λ	training	—
choose λ	validation	training error always picks the most flexible
report the error	test	tuning on the test set makes it optimistic again

Choosing the best of many settings on a set of cases fits that set's noise. The test set must stay untouched until the very end.

K-Fold Cross-Validation: Reuse the Training Set ★

One validation set wastes data and depends on which cases land in it. Instead, let every training case take a turn $(K = 5)$:



1. In round b, fit on the four blue folds and predict the orange fold.
2. Average the five fold errors: the CV error of one value of λ.
3. Repeat for every setting; choose the one with the smallest CV error.
4. Refit on the whole training set with that setting. Score it on the test set, once.

Every training case is predicted exactly once, by a model that never saw it.

Choosing λ by Cross-Validation on the GSS ★

The recipe of the previous slide, for λ. Lasso, 243 terms, 2,456 training cases in 5 folds of about 491 (figs/w12_690s.R, C):

left-out fold	$\lambda = 0.35$ (0 terms)	$\lambda = 0.024$ (42 terms)	$\lambda = 0.0014$ (179 terms)
1	0.853	0.665	0.695
2	0.930	0.708	0.748
3	0.948	0.759	0.813
4	0.784	0.537	0.568
5	0.811	0.591	0.670
CV-MSE	0.865	0.652	0.699

- Across a row: in every fold the middle λ predicts best. Too large keeps nothing (bias); too small keeps 179 terms (variance).
- Down a column: folds differ (0.54 to 0.76), so we average. `cv.glmnet` does this over 100 values of λ.

Cross-Validation: Two Rules ★

- $K = 5$ or 10 is standard: leave-one-out ($K = n$) has low bias but high variance and is expensive.
- Everything data-dependent happens inside each round, on the $K - 1$ training folds only, not once on the whole training set: variable selection, standardization, imputation. So the selected variables may differ slightly from fold to fold; that is correct. Selecting on all the data first is a serious leak (Lab 12 shows it).
- With dependence (panels, clusters, time) split on the independent unit, not on rows. Same rule as the bootstrap in Week 2.
- If λ is chosen by CV, the CV error at that λ is optimistic: report error on a test set never used for tuning.

Why the U-Shape? Bias and Variance ★

The CV error fell, then rose, as λ shrank. Why? Let $f(x)$ be the average of Y among everyone with $X = x$, the best possible prediction. Our $\hat{f}(x)$ changes from sample to sample:

$$\text{Bias}(\hat{f}(x)) = E[\hat{f}(x)] - f(x) \qquad \text{average prediction minus the truth}$$

$$\text{Var}\,(\hat{f}(x)) = E\Big[(\hat{f}(x) - E[\hat{f}(x)])^2\Big] \qquad \text{scatter around that average}$$

$$\text{MSE}(x) = E\Big[(\hat{f}(x) - f(x))^2\Big] \qquad \text{average squared distance from the truth}$$

- **Bias**: wrong on average. Rigid rules (large λ, few terms) have high bias.
- **Variance**: unstable. Flexible rules (small λ, many terms) have high variance.
- Expectations are over repeated *samples*: they describe a method, not one fit.

Decomposing the MSE ★

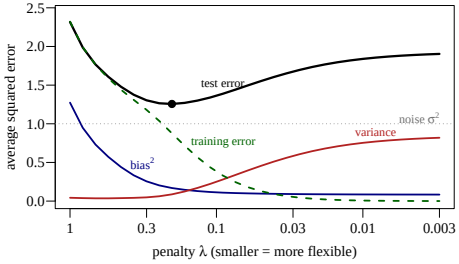
Same idea as the bandwidth in Week 9:

$$\begin{aligned}
\text{MSE}(x) &= E\Big[\big((\hat{f}(x) - E[\hat{f}(x)]) + (E[\hat{f}(x)] - f(x))\big)^2\Big] && \text{add and subtract } E[\hat{f}(x)] \\
&= E\Big[\big(\hat{f}(x) - E[\hat{f}(x)]\big)^2\Big] + \big(E[\hat{f}(x)] - f(x)\big)^2 && \text{expand the square;} \\
&\quad + 2\,\big(E[\hat{f}(x)] - f(x)\big)\,\underbrace{E\big[\hat{f}(x) - E[\hat{f}(x)]\big]}_{=0} && \text{the bias is a constant} \\
&= \text{Var}\left(\hat{f}(x)\right) + \text{Bias}\big(\hat{f}(x)\big)^2
\end{aligned}$$

- Average squared error = spread around the average prediction + distance of that average from the truth.
- For any approximation fitted on a sample, there is a trade-off between the two.

The Tradeoff in One Picture ★

Lasso on simulated data with 400 regressors and 100 cases, many samples; error averaged over new people (figs/w12_lasso_bv.R):



- Large λ (left): few terms, predictions steady but wrong on average. Bias dominates.
- Small λ (right): many terms, predictions jump from sample to sample. Variance dominates.
- Error on new people = noise + bias^2 + variance is lowest in between. Training error keeps falling, so it cannot find that point.

So Accept a Little Bias ★

- Weeks 1–2 prized unbiasedness: Gauss–Markov ranks estimators *within* the unbiased class. For prediction the target is total error, bias^2 + variance.
- OLS with 243 terms and 300 cases keeps bias low and lets variance explode (test R^2 −11.8). Shrinking gives up a little bias for a large cut in variance.
- On the GSS, CV error 0.865 with λ large (bias dominates), 0.699 with λ small (variance dominates), 0.652 in between.
- Every method after this has the same bargain and its own tuning parameter (depth for trees, size and training time for networks), chosen the same way: CV on the training set, the test set once, at the end.

Lasso Caveats ★

- The estimates shrink toward zero relative to the unpenalized regression: *shrinkage bias* or *regularization bias*. Lasso is biased by design.
- Lasso will not generally select the "right" set of variables.
- It tends to exclude variables with small but nonzero true coefficients (those with $|\hat{b}_j| \leq \lambda/(2n)$).
- It tends to fail to select the right variables when the X_i are *correlated* (next slide).
- So: do not interpret the selected set as "the variables that matter", and do not report p-values from a naive refit on the selected set. The selection invalidates them. Valid inference after selection is Week 13's job.

Lasso Caveats: Correlated Regressors

- Suppose X_1 has coefficient $\beta_1 = 0$ but is highly correlated with $X_2, \ldots, X_k$, whose coefficients are nonzero.
- The marginal predictive benefit of X_1 can be very high when $X_2, \ldots, X_k$ are *not* in the model, because X_1 stands in for all of them; the marginal benefit of any single one of $X_2, \ldots, X_k$ is then low.
- X_1 may enter with a nonzero coefficient while all of $X_2, \ldots, X_k$ are excluded.
- This inability to select *exactly* the right regressors is not special to lasso; every variable-selection procedure shares it.

Next slide: what this means on the GSS.

Selection Is Not Importance, on the GSS

Lasso on the 23 main-effect columns, 100 bootstrap samples, λ by the 1-se rule (path slide) (figs/w12_gss.R):

variable	share of samples selected	variable	share
education, age, female, ever married	1.00	parents' education	0.87
vocabulary score	0.99	Black	0.85
hours worked	0.97	Mountain, Pacific region	0.34
year 2022	0.90	number of children	0.09

- Strong predictors are kept every time; weaker ones (the regions) flip in and out: the selected set has sampling variability (Mullainathan and Spiess 2017).
- Children: dropped in 91% of samples. Yet the forest ranks it 7th: a forest can use it through interactions, a main-effect lasso cannot (figs/w12_importance.R).
- Two tools, two stories, same data. Neither ranking measures how much a variable matters.

Post-Lasso ★

- One way to undo the *shrinkage bias*: refit OLS using only the regressors whose lasso coefficients are nonzero.
- This is "least squares post lasso", or *post-lasso*:

$$\hat{\beta}^{\text{post}} = \underset{b \in \mathbb{R}^p}{\arg\min} \sum_{i=1}^{n} \left(Y_i - \sum_j X_{ij} b_j\right)^2 \quad \text{such that } b_j = 0 \text{ if } \hat{\beta}_j^{\text{lasso}} = 0$$

- Lasso chooses *which* variables; OLS chooses *how large*.
- Removing the shrinkage of the nonzero coefficients sometimes improves prediction, and sometimes does not.

Post-Lasso on the GSS

Standardized coefficients, lasso on the 23 main effects (λ by the 1-se rule, path slide), then OLS on the 9 kept (figs/w12_690s.R, D):

	educ	age	female	Black	vocab.	par. educ	hours	married	2022
lasso	0.251	0.103	−0.121	−0.017	0.043	0.024	0.050	0.058	0.028
post-lasso	0.286	0.155	−0.174	−0.058	0.061	0.070	0.101	0.098	0.087

- Every lasso coefficient is smaller in size: the female gap grows from −0.121 to −0.174 once the shrinkage is removed.
- Test R^2 on the 243-term design: CV lasso (45 terms) 0.261 → post-lasso 0.246. Removing shrinkage does not always help prediction.

Trees

From Linear to Nonlinear Models ★

- We want to predict Y_i from p regressors X_i.
- So far the prediction rule was *linear*: OLS, lasso, ridge. You choose the variables, and an interaction exists only if you type it.
- In reality the pattern is rarely linear; it has thresholds, curves, and interactions.
- Two classes of flexible methods find them on their own: **tree-based methods** and **neural networks**.
- Both are commonly used as the *nuisance* learners inside double machine learning (Week 13).

Regression Trees ★

- Partition the regressor space (where X_i takes values) into a set of *rectangles*, and fit a simple model within each.
- The splitting rules can be drawn as a tree, hence *decision trees*.
- The most common choice fits a *constant* in each rectangle, so the unknown function is approximated by a *step function*. With J regions $R_1, \ldots, R_J$:

$$f(X_i) = \sum_{j=1}^{J} \beta_j \mathbb{1}(X_i \in R_j),$$

  where β_j is a constant for region j and $\mathbb{1}(\cdot)$ the indicator.
- It is a regression on J dummies, one per leaf, whose definition the data choose.

Each Leaf Predicts Its Average ★

Given a partition, choose the constants to minimize the in-sample sum of squares:

$$\hat{\beta} = \underset{b_1,\ldots,b_J}{\arg\min} \sum_{i=1}^{n} \Big(Y_i - \sum_{j=1}^{J} b_j \, \mathbb{1}(X_i \in R_j) \Big)^2$$

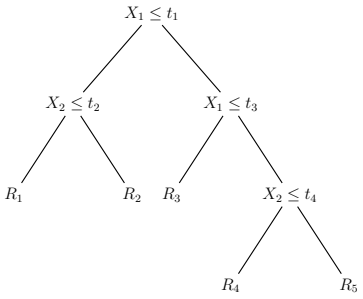
$$= \underset{b_1,\ldots,b_J}{\arg\min} \sum_{j=1}^{J} \sum_{i \in R_j} (Y_i - b_j)^2 \qquad \text{each } i \text{ is in exactly one region}$$

The problem separates by region. For region j:

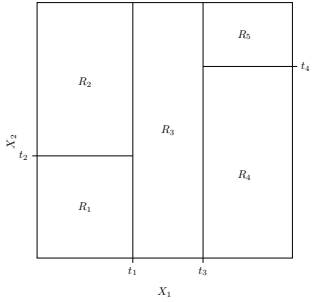
$$-2 \sum_{i \in R_j} (Y_i - b_j) = 0 \quad \Longrightarrow \quad \hat{\beta}_j = \frac{1}{n_j} \sum_{i \in R_j} Y_i = \bar{Y}_{R_j} \qquad n_j \text{ cases in } R_j$$

- The prediction in a leaf is the average outcome of the training cases in it, exactly the dummy-regression result of Week 1.
- The hard part is choosing the rectangles.

Recursive Binary Splitting ★



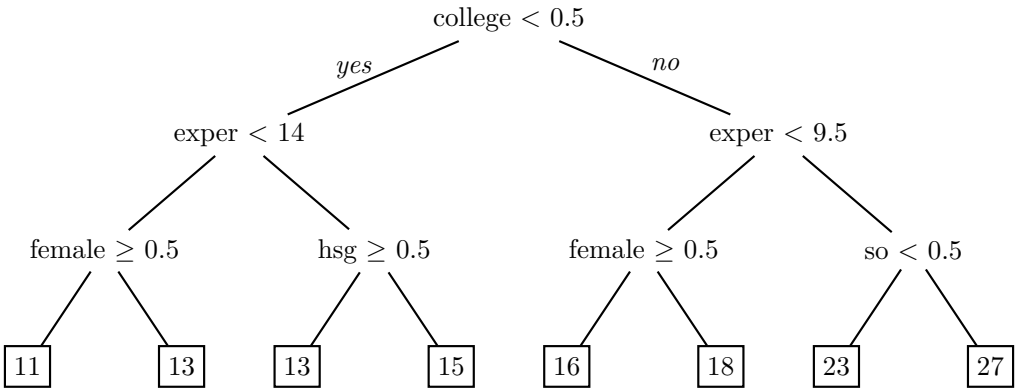
The tree: each node asks one yes/no question



The same tree as rectangles in (X_1, X_2)

Every split cuts one existing rectangle in two along one variable. The leaves are the final rectangles, each with its own constant.

A Depth-3 Tree for Wages



Depth 3: three questions from the root to any leaf, at most $2^3 = 8$ leaves (CPS wage data).

Growing a Regression Tree ★

- The cut points are chosen from the data, not fixed in advance.
- To keep computation tractable we split recursively, two children at a time.
- At each node, consider every predictor $X_1, \ldots, X_p$. For predictor X_j and threshold s, form the binary split

$$R_L = \{i : X_{ij} \leq s\}, \qquad R_R = \{i : X_{ij} > s\}$$

- Compute the sum of squared errors after the split,

$$\text{SSE}(j, s) = \sum_{i \in R_L} \left(Y_i - \bar{Y}_L\right)^2 + \sum_{i \in R_R} \left(Y_i - \bar{Y}_R\right)^2,$$

  with $\bar{Y}_L, \bar{Y}_R$ the mean outcomes in the two children.
- Choose the (j, s) with the smallest $\text{SSE}(j, s)$: the largest reduction in MSE.

One Split by Hand

Six people, one predictor (years of schooling) (figs/w12_690s.R, F):

schooling	10	12	12	14	16	18
log earnings	9.6	9.8	10.0	10.4	10.8	11.0

No split: everyone gets the mean 10.27, SSE = 1.573. Try every cut:

cut	left: n, mean	right: n, mean	$\text{SSE}_L + \text{SSE}_R$	total
≤ 11	1, 9.60	5, 10.40	$0.000 + 1.040$	1.040
≤ 13	3, 9.80	3, 10.73	$0.080 + 0.187$	0.267
≤ 15	4, 9.95	2, 10.90	$0.350 + 0.020$	0.370
≤ 17	5, 10.12	1, 11.00	$0.928 + 0.000$	0.928

Split at 13 (the high-school/college line): the SSE falls by 83%. Prediction: 9.80 with at most 12 years, 10.73 with more. With more predictors, the same search runs over every variable.

The First Split on the GSS

The 23 main-effect predictors, 2,456 training cases; `rpart` searches every variable and every cut (figs/w12_690s.R, F2):

node	n	mean log earnings	SSE
root (everyone)	2,456	9.94	2,129.5
schooling $<$ 15.5 years	1,407	9.66	1,137.5
schooling $\geq$ 15.5 years	1,049	10.31	733.0

- SSE falls from 2,129.5 to $1{,}137.5 + 733.0 = 1{,}870.5$, a 12.2% cut: the training R^2 of a one-split tree. The cut is the college line.
- The next splits differ by branch: without a degree, the tree splits next on *female*; with one, on *age* (at 32.5). Sex matters first in one branch, age in the other: an **interaction** nobody typed.

Growing a Regression Tree: Depth ★

- Repeat the search inside each child, recursively.
- Splits at level 2 or deeper may use different variables and cut points in different branches; that is how trees create interactions and nonlinearities.
- Stop when a prespecified depth is reached, or (more commonly) when a node would have fewer than a minimum number of observations.
- **Why depth matters**: the deeper the tree, the smaller the bias, but the noisier the prediction, since fewer observations sit in each leaf to estimate its mean. The bias–variance tradeoff again.

Depth Sets the Bias–Variance Tradeoff, on the GSS ★

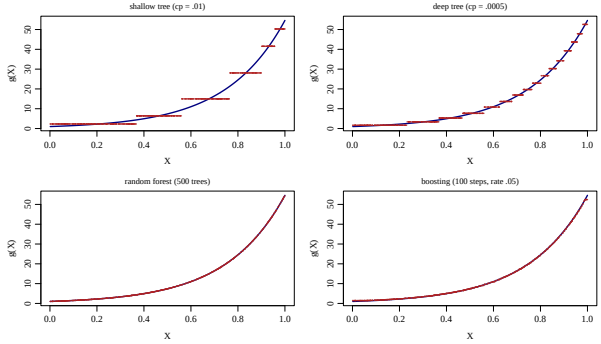
Trees grown to a fixed depth, no other stopping (figs/w12_690s.R, F2):

depth	1	2	3	5	8	12	20
leaves	2	4	8	31	180	708	1,938
training R^2	0.122	0.168	0.215	0.313	0.540	0.769	0.975
test R^2	0.132	0.179	0.231	0.218	0.074	−0.101	−0.368

- Training R^2 rises with every level; at depth 20 the tree memorizes the sample (1,938 leaves for 2,456 people).
- Test R^2 peaks around depth 3, then collapses below zero: worse than predicting the mean.
- We find the right depth by validation: a train/test split or CV.

A Tree Fitting a Curve

The noiseless curve $Y = e^{4X}$, 1,000 points; blue = truth, red = fit (figs/w12_fitml.R):



Top: a shallow tree (6 leaves) is a coarse staircase; a deep tree (16 leaves) tracks the curve. Bottom: averaging many trees (forest) or adding many small ones (boosting) smooths the steps away. Next slides.

Pruning a Tree ★

- Trying every possible tree structure and cross-validating each is not feasible.
- Better: grow a large tree T_0 (stop only when leaves are small), then *prune* it back to a *subtree* $T \subset T_0$.
- *Cost-complexity pruning*: for a penalty α, choose the subtree that minimizes

$$C_\alpha(T) = \sum_{m=1}^{|T|} \sum_{i \in R_m} \left(Y_i - \bar{Y}_{R_m}\right)^2 + \alpha\,|T|,$$

  where $|T|$ is the number of leaves.
- As in lasso, the penalty trades complexity against in-sample fit: each extra leaf must lower the SSE by more than α.
- α, and with it the tree, is chosen by a validation set or CV.

GSS: the CV-pruned tree reaches test R^2 0.215, below OLS (0.251). One tree is too rough a step function for a mostly smooth relationship.

Bagging ★

- Even tuned, one tree has *high variance*, especially in small samples.
- *Bootstrap aggregation*, or *bagging*, is a general way to reduce the variance of a learning method.
- Recall: for independent $Z_1, \ldots, Z_B$, each with variance σ^2, $\text{Var}(\bar{Z}) = \sigma^2/B$. Averaging cuts variance.
- Draw B bootstrap samples; grow a tree on each, *deep* and *unpruned*: low bias, high variance. Average:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{\,b}(x)$$

- The average keeps the low bias and sheds much of the variance. Hundreds or thousands of trees are typical.

GSS: one deep tree, test R^2 -0.264; 500 bagged deep trees, 0.268 (figs/w12_690s.R, G).

Random Forests ★

- Random forests improve on bagging by *decorrelating* the trees.
- As in bagging, grow trees on bootstrap samples. But at each split, consider only *a random sample of m predictors* out of the p, a fresh sample at every split; typically $\sqrt{p}$ rounded down (GSS: $\sqrt{23} = 4.8$, so $m = 4$).
- **Why**: if one strong predictor dominates the first split, every bagged tree splits on it first, and the trees look alike. Averaging near-copies removes little variance, and the other predictors' information goes unused.
- On the GSS, education is that predictor: it is the root split (slide on the first GSS split).
- Forcing splits to consider different subsets gives different trees, and their average varies less.

Why Decorrelating Trees Works ★

B trees, each with variance σ^2 at a point, pairwise correlation ρ:

$$\begin{aligned}
\operatorname{Var}\Big(\frac{1}{B}\sum_{b=1}^{B} T_b\Big) &= \frac{1}{B^2}\Big(\sum_b \operatorname{Var}(T_b) + \sum_{b\neq b'} \operatorname{Cov}(T_b, T_{b'})\Big) && \text{variance of a sum} \\
&= \frac{1}{B^2}\big(B\sigma^2 + B(B-1)\rho\sigma^2\big) && B(B-1) \text{ ordered pairs} \\
&= \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2 && \text{rearrange}
\end{aligned}$$

- More trees kill only the second term. The first, $\rho\sigma^2$, is a floor. With $\sigma^2 = 1$, $B = 500$: $\rho = 0.6$ gives 0.601; $\rho = 0.3$ gives 0.301.
- GSS: two trees' test predictions correlate 0.34 when bagging, 0.27 in the forest; test R^2 0.268 vs. 0.272: little gain here (figs/w12_690s.R, G).
- Out-of-bag error: each tree skips about a third of the data ($(1-1/n)^n \approx e^{-1}$). GSS: OOB MSE 0.626, test MSE 0.645.

Which Variables Does the Forest Use? Two Measures ★

A forest averages 1,000 trees: no single tree to read, no coefficient. Instead ask *how much the predictions depend on each variable.*

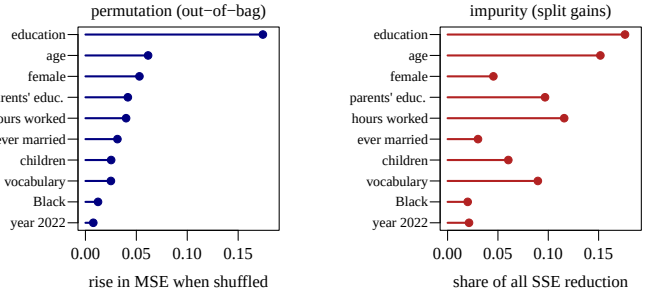
1. Permutation importance. Shuffle one column, which breaks its link with Y and leaves its distribution unchanged. Predict again and record how much the error rises.

- GSS forest, 1,053 test cases: test MSE 0.645. Shuffle education: 0.809. Its importance = 0.164.
- `ranger` does this tree by tree, on each tree's out-of-bag cases (`importance = "permutation"`).

2. Impurity importance. Every split cuts the SSE, as in the earlier split by hand (the root split on schooling cut it by 259.0). Credit each cut to the variable that was split on, then add the cuts up within a tree and average over the trees.

Impurity: `importance = "impurity"`, free while growing the trees. Same forest as the benchmark table: 2,456 training cases, 23 columns (figs/w12_importance.R).

Variable Importance on the GSS



- Both put education first, then age. They disagree on *female*: 3rd by permutation, 7th by impurity. Female takes 2 values, age 40: more cut points, more chances to win a split by luck, so impurity favours many-valued variables (Strobl et al. 2007).
- Use permutation, and ask for it; it is not R's default. `randomForest` reports impurity unless `importance = TRUE`; `ranger` reports none unless `importance = "permutation"`.

Reading Importance With Care

Correlated variables share credit. Age and ever married correlate at 0.36. Rise in test MSE when shuffled (figs/w12_importance.R):

shuffled	rise in test MSE
age alone	0.057
ever married alone	0.017
sum of the two	0.074
age and ever married *together*	0.090

- Shuffle one and the forest leans on the other, so each looks less important alone than the pair is jointly. Shuffle related variables as a group.
- The ranking depends on the measure: hours is 3rd by impurity, 4th by test-set permutation, 5th by out-of-bag permutation.
- Importance describes *the model's predictions*, not the world. A high score says the forest relies on the variable, not that changing it changes earnings (Section Interpretation).

Tuning a Random Forest

Three tuning parameters in `ranger`; judge them by the **out-of-bag** MSE, which needs no validation fold (figs/w12_rf_tune.R):

- `mtry` (m): how many predictors each split may try. Default: $\sqrt{p}$ rounded down; here $\sqrt{23} = 4.8$, so 4.
- `min.node.size`: fewest cases in a leaf. Default 5; larger gives shallower, smoother trees.
- `num.trees`: more never hurts (OOB MSE 0.815 with 10 trees, 0.626 with 1,000).

OOB MSE	`min.node.size`				
`mtry`	1	5	20	50	100
1	0.740	0.740	0.742	0.744	0.749
3	0.630	0.629	0.629	0.638	0.648
5	0.626	0.623	0.623	0.625	0.631
8	0.633	0.629	0.626	0.623	0.627
23 (bagging)	0.663	0.656	0.642	0.635	0.634

`mtry` matters most; leaf size barely, once m is sensible. Test R^2, touched once: default 0.272, tuned ($m = 8$, leaf 50) 0.286, bagging 0.264. Forests work well at their defaults.

In the Literature: What Predicts Relationship Quality?

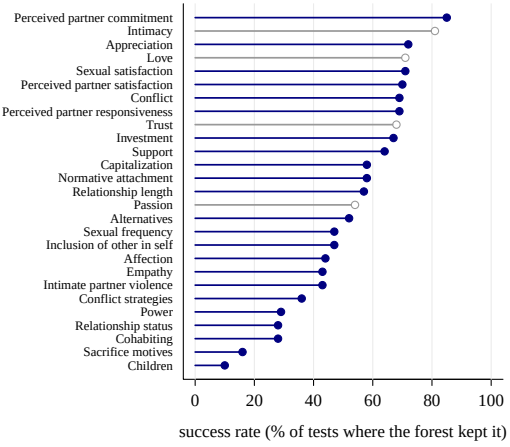
The question. Joel et al. (2020): researchers have proposed hundreds of variables that shape relationship quality. How predictable is it, and which variables actually predict it?

The data. 43 longitudinal couples studies from 29 labs: 11,196 couples, 2,413 measures at baseline, followed for about 14 months.

The method. A random forest in each study (R's `randomForest`, 5,000 trees, $m = p/3$), variance explained judged on out-of-bag cases.

Which predictors? The `VSURF` package ranks predictors by **permutation importance** (how much the error rises when a variable is shuffled), drops the weak ones, then adds the rest one at a time and keeps those that lower the out-of-bag error. Results are pooled across the 43 studies.

Joel et al.: Which Constructs the Forests Kept



How often `VSURF` kept each construct across the 43 studies: a selection rate, not an importance score.

- Top: perceived partner commitment (85%), appreciation, sexual satisfaction. Bottom: sacrifice motives (16%), children (10%).
- Hollow gray: intimacy, love, trust, passion. Left out of the main models: they may be part of relationship quality itself.
- A rank, not an effect: no size, no direction, no causation.

Boosted Trees ★

- Bagging and random forests reduce *variance* from low-bias estimates (deep trees) by repeated sampling. *Boosting* reduces *bias* from low-variance estimates (shallow trees) by repeatedly fitting residuals.
- Fit a simple rule (a shallow tree), take the *residuals*, fit another shallow tree to those residuals, and so on.
- Each step fits what the previous steps missed, so each step reduces the remaining approximation error.
- Stop at a stopping criterion. The overall prediction is the sum of the rules from all steps.

Boosted Trees: the Algorithm ★

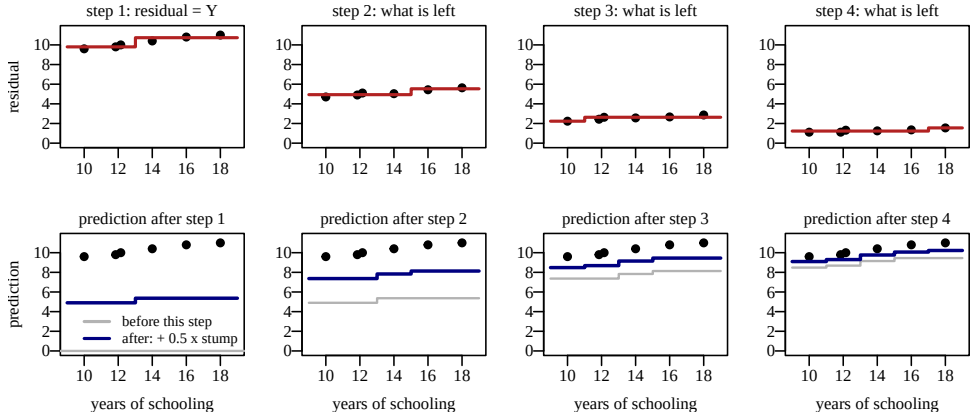
1. Initialize the residuals: $R_i := Y_i$, $i = 1, \ldots, n$.
2. For $j = 1, \ldots, J$:
   - fit a tree-based rule $\hat{f}_j(X_i)$ to the data $(X_i, R_i)_{i=1}^n$;
   - update the residuals, $R_i := R_i - \lambda \hat{f}_j(X_i)$, where λ is the *learning rate*.
3. Output the boosted rule

$$\hat{f}(X_i) := \sum_{j=1}^{J} \lambda \hat{f}_j(X_i).$$

Choices: the tree depth, the number of steps J, and the learning rate (a common default is $\lambda = 0.1$), usually by CV. In practice one starts from the mean rather than from 0.

Boosting by Hand: Start From Zero

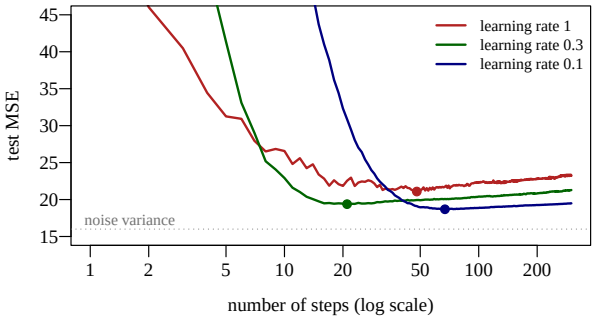
Six people; stumps; $\lambda = 0.5$; guess 0, so $R_i = Y_i$ (figs/w12_boost_steps.R).



Person with 10 years ($Y = 9.6$): prediction $0 \to 4.90 \to 7.37 \to 8.48 \to 9.10$. Each step adds half of what is still missing, so the gap halves: $9.6 \to 4.70 \to 2.23 \to 1.12 \to 0.50$.

Small Steps Predict Better on New Cases

$Y = e^{4X} +$ noise (sd 4); 100 training, 5,000 test cases; stumps (figs/w12_boost_steps.R):



- Lowest test MSE: $\lambda = 1$: 21.1 at step 48; $\lambda = 0.3$: 19.4 at step 21; $\lambda = 0.1$: 18.7 at step 67.
- A big rate peaks early, peaks worse, then climbs as later stumps fit noise. A small rate needs more steps but gets closer to the noise floor of 16.

Boosted Trees: How Many Steps

- The number of steps J matters.
- Each step fits the part of the outcome the previous steps left unexplained, so the in-sample error can only fall (weakly) with each step: by hand, the 10-year person's gap went 9.6, 4.70, 2.23, 1.12, 0.50.
- Too many steps and the model overfits: eventually it fits the noise.
- Too few steps leave substantial bias.
- In practice, stop once CV-MSE no longer improves.
- A small learning rate with many steps usually beats a large rate with few: many cautious steps rather than a few bold ones.
- A popular implementation is `xgboost` (Chen and Guestrin 2016).

In the Literature: Machine Predictions and Bail

The question. Kleinberg et al. (2018): judges decide pretrial release by predicting crime if released. Could an algorithm predict better?

The data and method. New York City arrests 2008–13; 554,689 cases in the working sample; gradient boosted decision trees predict crime if released. Judges release 73.6%; 11.3% of the released commit a crime before trial.

The results (held-out test set):

- Same release rate as the judges: crime falls 24.7%.
- Same crime rate: detention falls from 26.4% to 15.3% (a 41.9% cut).
- Both can be achieved while reducing racial disparities in detention.

Bagging, Random Forests, Boosting ★

- **Bagging**: fit trees on bootstrap resamples and average. Averaging B nearly independent estimates cuts variance by roughly $1/B$ without adding bias. But bootstrap trees are *correlated*, they all split on the same dominant predictor first, which caps the gain.
- **Random forests**: bag, but at each split consider only a random subset of m predictors. Decorrelating the trees is what makes the averaging work. m is the main tuning parameter, alongside node size. Out-of-bag error is a free cross-validation estimate.
- **Boosting**: fit trees *sequentially*, each on the residuals of the current ensemble, adding a shrunken version: $\hat{f} \leftarrow \hat{f} + \lambda \hat{t}$. Attacks *bias* rather than variance, and with a small learning rate λ and many trees is usually the strongest tabular predictor available.

Bagging and random forests reduce variance; boosting reduces bias. That is the cleanest way to keep them straight.

One Benchmark: Log Earnings in the GSS

70% training / 30% test, $n = 3{,}509$; 23 main-effect columns, 243 in the expanded design; tuning by CV inside the training set (figs/w12_gss.R):

test R^2	training $n = 2{,}456$	training $n = 300$
training mean	0.000	−0.014
OLS, 23 main effects	0.251	0.188
OLS, 243 terms	0.200	−11.752
ridge, 243 terms	0.253	0.180
lasso, 243 terms	0.261	0.188
elastic net, 243 terms	0.261	0.197
pruned tree	0.215	0.123
random forest	0.272	0.185
boosting	0.285	0.156
neural network (1 layer)	0.259	0.137

Large n: ensembles win, modestly. Small n: regularization is everything; the unpenalized 243-term model is far worse than predicting the mean. Most of the variation in earnings stays unpredicted.

In the Literature: the Fragile Families Challenge

The question. Salganik et al. (2020): how predictable are life outcomes, given rich data from birth to age 9? **The design.** A mass collaboration: 4,242 families, 12,942 variables; outcomes at age 15 released for half the families; 160 teams submitted predictions, scored by test R^2 on families whose outcomes no team saw.

outcome	best test R^2	outcome	best test R^2
material hardship	0.23	eviction	0.05
child's GPA	0.19	caregiver job training	0.05
child's grit	0.06	caregiver layoff	0.03

- The best of 160 teams was only slightly better than a four-variable linear benchmark chosen by a domain expert: three facts about the mother at birth (race, marital status, education) plus the same outcome measured at age 9.
- Errors depended on the *family*, not on the method: the same families were mispredicted by everyone.

Combining Learners: Stacking ★

No method wins everywhere: lasso won the small-sample GSS race among linear models, the forest won overall. **Stacking** lets the data choose a mix (van der Laan, Polley, and Hubbard 2007):

1. Fit each candidate $\hat{f}_1, \ldots, \hat{f}_M$ with K-fold cross-validation; keep the out-of-fold predictions $\hat{f}_m^{(-k)}(X_i)$.
2. Regress Y_i on the out-of-fold predictions with non-negative weights summing to one: $\min_a \sum_i \left(Y_i - \sum_m a_m \hat{f}_m^{(-k)}(X_i)\right)^2$.
3. Predict with $\sum_m \hat{a}_m \hat{f}_m$, each refit on all data.

- Out-of-fold predictions keep a learner that overfits from getting weight it did not earn.
- Asymptotically at least as good as the best single candidate (the "super learner" oracle result). Next week it supplies the nuisance functions inside DML.

Neural Nets

Neural Networks: the Idea ★

Plainly: a flexible nonlinear function of many inputs. Like a tree, it finds curves, thresholds, and interactions you did not type in, but it gets there differently:

	trees and forests	neural networks
how	cut the inputs into boxes, one variable at a time; predict the average in each box	make new variables, each a weighted mix of *all* inputs bent by a smooth curve; then regress Y on them
shape	steps	smooth surfaces
best on	tables of survey variables (the GSS)	text, images, sound: many raw inputs whose useful features are combinations (pixels → edges → cars)

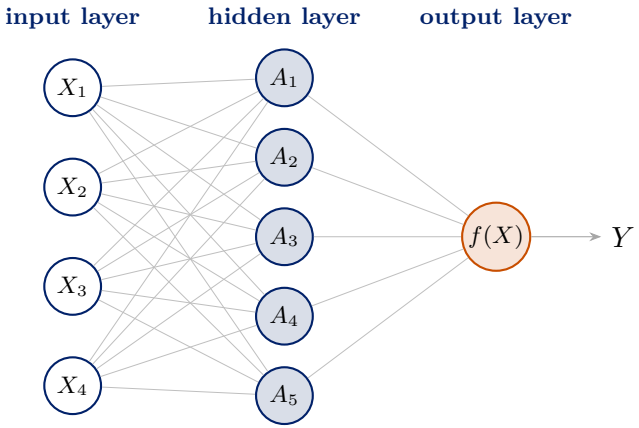
Uses in this course: measuring from text and images (Gebru et al. 2017, end of this section; word vectors, next section), and predicting outcomes and treatments inside causal estimators (Week 13).

Single Layer Neural Network ★

- A neural network takes the p inputs $X_i = (X_{1i}, \ldots, X_{pi})$ and builds a nonlinear function $f(X_i)$ to predict Y_i.
- The p features make up the units of the *input layer*.
- Every input feeds into each of K hidden units, the *hidden layer*.
- A single-layer network has the form

$$f(X_i) = \beta_0 + \sum_{k=1}^{K} \beta_k A_k = \beta_0 + \sum_{k=1}^{K} \beta_k \, h_k(X_i)$$
$$= \beta_0 + \sum_{k=1}^{K} \beta_k \, \sigma\Big(w_{k0} + \sum_{j=1}^{p} w_{kj} X_{ji}\Big)$$

Single Layer Neural Network: Picture ★



Each line from an input to a hidden unit carries a weight w_{kj}; each line from a hidden unit to the output carries a weight β_k.

Two Steps: Hidden Units, Then Output ★

1. Each of the K **hidden units** (*neurons*) takes a weighted sum of the inputs and bends it. The number it outputs is its *activation* A_k:

$$A_k = h_k(X_i) = \sigma\Big(w_{k0} + \sum_{j=1}^{p} w_{kj} X_{ji}\Big),$$

   where $\sigma(\cdot)$ is the *activation function*, a fixed bend chosen in advance. Neuron = the node; activation = its output. (Software sometimes lists σ as its own "layer"; it is part of each neuron.)
2. The K activations feed into the output layer, a linear regression on the learned features:

$$f(X_i) = \beta_0 + \sum_{k=1}^{K} \beta_k A_k$$

β_k and w_{kj} are called *weights*; β_0 and w_{k0} are called *biases* (intercepts, not the bias of the bias–variance tradeoff).

Activation Functions ★

- The *sigmoid* function

$$\sigma(z) = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}},$$

  the same function the logit uses to turn a linear index into a probability (Week 4).
- The modern default is the *ReLU* (*rectified linear unit*): $\sigma(z) = (z)_+ = \max(0, z)$.

z	-1	0.5	2
sigmoid	0.27	0.62	0.88
ReLU	0	0.5	2

- ReLU is cheap to compute and store, and its derivative is simple (0 or 1). It thresholds at 0, but the intercept w_{k0} moves the threshold, like a neuron that fires only above a threshold.

Why the Activation Must Be Nonlinear ★

Suppose $\sigma(z) = z$ instead:

$$f(X) = \beta_0 + \textstyle\sum_k \beta_k(w_{k0} + \sum_j w_{kj}X_j) \qquad \text{substitute the identity}$$

$$= \underbrace{(\beta_0 + \textstyle\sum_k \beta_k w_{k0})}_{\text{intercept}} + \textstyle\sum_j \underbrace{(\sum_k \beta_k w_{kj})}_{\text{slope on } X_j} X_j \qquad \text{regroup}$$

- A linear network of any width or depth is one linear regression. All the flexibility comes from σ.
- With ReLU and the intercept w_{k0}, each unit is a hinge at its own location: ReLU networks are piecewise-linear functions.

Why the Bend Makes a Network Flexible ★

- Each hidden unit takes a weighted sum of the inputs and **bends** it (S-curve, ReLU). The output adds the bent pieces with weights.
- **Without the bend**, a weighted sum of weighted sums is still a weighted sum: the network is just a linear regression, however many units it has.
- **With the bend**, adding pieces that switch on at different places draws curves, thresholds, and interactions nobody typed in. Example with $\sigma(z) = z^2$ and two units: $\frac{1}{4}(X_1 + X_2)^2 - \frac{1}{4}(X_1 - X_2)^2 = X_1 X_2$, an interaction.
- **More units, finer shapes**: with enough hidden units a network can approximate any smooth function (Hornik, Stinchcombe, and White 1989). That says it is possible, not how many units or how to find the weights (training, next).

Number of Parameters in a Single Layer Network

With p inputs and K neurons:

- input to hidden layer: $(p+1) \times K$, that is $p \times K$ weights and K biases;
- hidden to output layer: $K+1$, that is K weights and 1 bias.

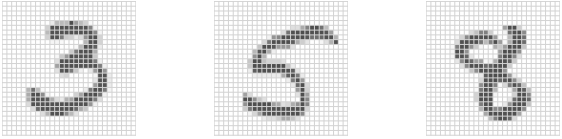
To fit, minimize the loss (continuous Y_i) over all of them:

$$\underset{W \in \mathbb{R}^{(p+1)\times K};\ \beta \in \mathbb{R}^{K+1}}{\arg\min} \sum_{i=1}^{n} \left(Y_i - f(X_i)\right)^2$$

GSS: 23 inputs, 2 hidden units: $(23+1) \times 2 + (2+1) = 48 + 3 = 51$ parameters.

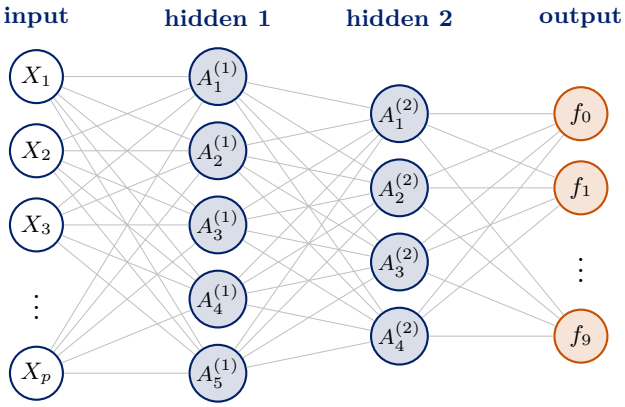
Multilayer Neural Networks ★

- Single-layer networks have limited power in practice; stacking layers can greatly improve prediction.
- Multilayer networks famously solved the *MNIST* handwritten-digit problem.



Each grayscale image has $28 \times 28 = 784$ pixels, each an 8-bit number (0–255) for how dark the pixel is.

Two-Layer Neural Network: Picture



Two hidden layers and several outputs, one per digit.

Two-Layer Neural Network

- MNIST: the input layer has $p = 784$ units; hidden layer L_1 has $K_1 = 256$ units, L_2 has $K_2 = 128$; the output layer has 10 units, the probabilities of the digits 0–9 from a *softmax* (the multinomial logit of Week 4).
- Each neuron $k = 1, \ldots, K_1$ in the first hidden layer:

$$A_k^{(1)} = h_k^{(1)}(X_i) = \sigma\Big(w_{k0}^{(1)} + \sum_{j=1}^{p} w_{kj}^{(1)} X_{ji}\Big)$$

- The second hidden layer treats the neurons of L_1 as its inputs:

$$A_\ell^{(2)} = h_\ell^{(2)}(X_i) = \sigma\Big(w_{\ell 0}^{(2)} + \sum_{k=1}^{K_1} w_{\ell k}^{(2)} A_k^{(1)}\Big), \qquad \ell = 1, \ldots, K_2$$

- Depth builds features of features (edges, then shapes, then digits).

Number of Parameters in a Two-Layer Network ★

With p inputs, K_1 and K_2 hidden neurons, and m outputs:

layer to layer	count	MNIST
input $\rightarrow$ first hidden	$(p+1) \times K_1$: pK_1 weights, K_1 biases	$785 \times 256 = 200{,}960$
first $\rightarrow$ second hidden	$(K_1+1) \times K_2$: K_1K_2 weights, K_2 biases	$257 \times 128 = 32{,}896$
second hidden $\rightarrow$ output	$(K_2+1) \times m$: K_2m weights, m biases	$129 \times 10 = 1{,}290$
total		235,146

- 235,146 parameters for 60,000 training images: more parameters than cases, and still it predicts well.
- Regularization and the fitting algorithm, not the parameter count, control the effective complexity.

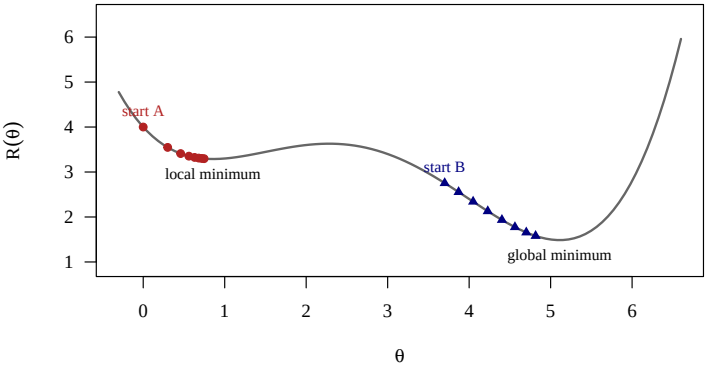
Fitting a Neural Network

- Take the single-layer network; the multilayer case works the same way.
- We minimize the squared loss over $(p+1)K$ input weights and biases and $K+1$ output ones.
- That is hard, for two reasons:
  - With a nonlinear activation, the loss is *non-convex*. Gradient $= 0$ is necessary but no longer sufficient for the global minimum; we can land in a local minimum or at a saddle point.
  - The sheer number of parameters: no closed form, no single calculation solves the gradient equations.
- Collect all parameters in one long vector θ, of length $(p+1)K + (K+1)$, and minimize

$$R(\theta) = \frac{1}{2}\sum_{i=1}^{n}\left(Y_i - f_\theta(X_i)\right)^2$$

by *gradient descent*, with *regularization* against overfitting.

Gradient Descent: Picture



- Two minima, one local, one global: where you start decides where you end.
- Is a local minimum good enough? For prediction, usually yes: the global minimum of training error would fit the noise (early stopping avoids it on purpose), and in large networks different local minima give similar error.

Gradient Descent

1. Start with an initial guess θ^0 for all parameters (small random numbers); set $t = 0$.
2. Find a small change δ such that $\theta^{t+1} = \theta^t + \delta$ *reduces* the loss: $R(\theta^{t+1}) < R(\theta^t)$.
3. Stop when the improvement is negligible, e.g. $|R(\theta^{t+1}) - R(\theta^t)| < \varepsilon$.

The step comes from the *gradient* of R at θ^t, with a small positive *learning rate* ρ:

$$\delta = -\rho \nabla R(\theta^t) = -\rho \left. \frac{\partial R(\theta)}{\partial \theta} \right|_{\theta=\theta^t}$$

- The gradient points uphill; we step the other way.
- Too large a ρ overshoots; too small crawls.

Gradient Descent by Hand

One parameter, three outcomes $Y = (1, 2, 6)$, the model predicts θ for everyone:

$$R(\theta) = \tfrac{1}{2}\sum_i (Y_i - \theta)^2, \qquad \frac{dR}{d\theta} = -\sum_i (Y_i - \theta), \qquad \theta^{t+1} = \theta^t + 0.1\sum_i (Y_i - \theta^t)$$

t	θ^t	$R(\theta^t)$	gradient
0	0.000	20.50	$-(1+2+6) = -9.00$
1	$0 - 0.1(-9.00) = 0.900$	13.62	-6.30
2	1.530	10.24	-4.41
3	1.971	8.59	-3.09
4	2.280	7.78	-2.16
5	2.496	7.38	-1.51

Each step closes 30% of the gap to the minimizer $\bar{Y} = 3$; the gradient shrinks as we approach it, so the steps shrink too (figs/w12_690s.R, I). Here R is convex; a network's is not.

The Gradient for One Observation ★

- Computing the gradient is feasible. Take the loss for one observation i; the total loss is the sum over the n observations:

$$\begin{aligned} R_i(\theta) &= \tfrac{1}{2}\big(Y_i - f_\theta(X_i)\big)^2 \\ &= \tfrac{1}{2}\Big(Y_i - \beta_0 - \sum_{k=1}^{K} \beta_k\, \sigma\Big(w_{k0} + \sum_{j=1}^{p} w_{kj} X_{ji}\Big)\Big)^2 \end{aligned}$$

- For short, write the linear index of hidden unit k as

$$z_{ik} = w_{k0} + \sum_{j=1}^{p} w_{kj} X_{ji}, \qquad A_{ik} = \sigma(z_{ik}).$$

Backpropagation: the Chain Rule ★

- For an output weight β_k:

$$\frac{\partial R_i}{\partial \beta_k} = \frac{\partial R_i}{\partial f_\theta(X_i)} \cdot \frac{\partial f_\theta(X_i)}{\partial \beta_k} = -\big(Y_i - f_\theta(X_i)\big) \cdot \sigma(z_{ik})$$

- For an input weight w_{kj}:

$$\begin{aligned}\frac{\partial R_i}{\partial w_{kj}} &= \frac{\partial R_i}{\partial f_\theta(X_i)} \cdot \frac{\partial f_\theta(X_i)}{\partial \sigma(z_{ik})} \cdot \frac{\partial \sigma(z_{ik})}{\partial z_{ik}} \cdot \frac{\partial z_{ik}}{\partial w_{kj}} \\ &= -\big(Y_i - f_\theta(X_i)\big) \cdot \beta_k \cdot \sigma'(z_{ik}) \cdot X_{ji}\end{aligned}$$

- Both contain the residual $Y_i - f_\theta(X_i)$: differentiation hands a share of the residual to each parameter through the chain rule. Compute f forward, pass the residual backward: *backpropagation*.
- Deep networks repeat the second line once per layer. If σ' is small (sigmoid in its flat tails), products of many small numbers vanish: one reason ReLU replaced sigmoid.

Stochastic Gradient Descent

- The gradient of the loss $R(\theta) = \sum_i R_i(\theta)$ is the sum of the n gradients ∇R_i. Dividing by n (the *average* loss) only rescales the step ρ, but keeps the step from growing with n; software uses the average:

$$\theta^{t+1} = \theta^t - \rho \frac{1}{n} \sum_{i=1}^{n} \nabla R_i(\theta^t)$$

  Every update uses all n observations: expensive.
- *Stochastic gradient descent* (SGD) uses only a random subsample B_t, a *mini-batch*, at each update:

$$\theta^{t+1} = \theta^t - \rho \frac{1}{|B_t|} \sum_{i \in B_t} \nabla R_i(\theta^t)$$

- One full pass through all observations is an *epoch*. At the start of each epoch the data are shuffled and cut into non-overlapping mini-batches.
- With $n = 1{,}000$ and batches of 10: 100 updates per epoch instead of one.

Dealing With Overfitting: Weight Decay

- As in high-dimensional regression, a massive number of parameters risks overfitting.
- A common fix is the ridge (ℓ_2) penalty on all weights and biases:[1]

$$R_i^{\text{ridge}}(\theta) = R_i(\theta) + \lambda \sum_j \theta_j^2$$

- SGD with a one-observation mini-batch then updates

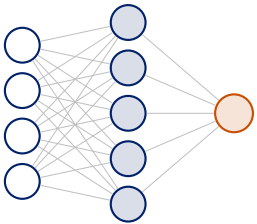
$$\theta^{t+1} = \theta^t - \rho\big(\nabla R_i(\theta^t) + 2\lambda\,\theta^t\big) = (1 - 2\rho\lambda)\,\theta^t - \rho\,\nabla R_i(\theta^t)$$

- Each step first shrinks every weight by the factor $1 - 2\rho\lambda$ (it "decays"), then steps downhill.

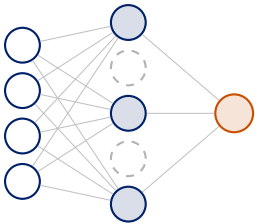
[1] $\sum_j \theta_j^2$ sums the squares of every entry of θ.

Dealing With Overfitting: Dropout

- Lasso-like penalties that zero out weights are less common in networks. Instead, *dropout learning* keeps units from becoming over-specialized.



full network

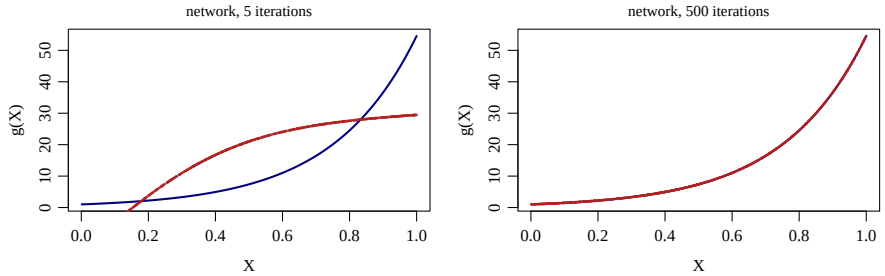


dropout network

- In each mini-batch, a fraction ϕ of a layer's units is dropped at random (their activations set to 0); the architecture stays intact.
- The rest are scaled by $1/(1-\phi)$, keeping the expected input unchanged ($\phi = 0.5$: survivors count double). No unit can rely on a particular other unit.

Dealing With Overfitting: When to Stop ★

1,000 simulated pairs (X_i, Y_i), one input and one output, $Y_i = e^{4X_i}$; one hidden layer of 10 units (31 numbers), optimizer stopped early or late (figs/w12_fitml.R):



- After 5 iterations the network is still a gentle curve; after 500 it tracks the truth. With noisy data, running too long fits the noise instead.
- **Early stopping**: watch the error on a held-out validation set; stop when it starts rising. For linear models, stopping gradient descent early acts like a ridge penalty.

Tuning a Neural Network

Three classes of hyperparameters must be tuned for a network to perform well:

- *Architecture*: number of hidden layers and units per layer.
- *Regularization*: the weight penalty λ and the dropout rate ϕ, possibly separately for each layer.
- *Optimization*: the SGD details, learning rate ρ, batch size, and number of epochs.

For large networks CV is often too expensive. Instead, hold out one validation set and monitor performance on it while tuning.

A Network on the GSS

One hidden layer (`nnet`), inputs standardized; hidden units $\{2, 5, 10, 20\}$ and weight decay $\{0.01, 0.1, 1\}$ by 5-fold CV; average of 5 random starts (figs/w12_gss.R).

- CV picks the smallest network: 2 hidden units, decay 0.1, 51 parameters.
- Test R^2 0.259: about lasso's (0.261), below random forests (0.272) and boosting (0.285). With 300 training cases: 0.137, below OLS.
- Nothing in 23 survey variables needs learned representations; the signal is mostly additive and smooth, and trees capture the few thresholds.

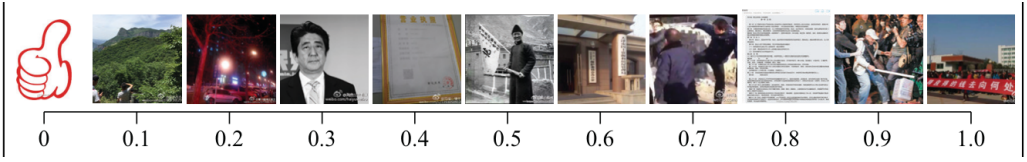
This matches the benchmark literature: on tables of data, tree ensembles still beat deep networks (Grinsztajn, Oyallon, and Varoquaux 2022). Networks win when the input is unstructured: text, images, audio.

In the Literature: Finding Protests in Social Media

The question. Zhang and Pan (2019): protest data usually come from newspapers. Where the press is controlled, as in China, most protests never reach the news. Can social media posts find them?

The data. 9.5 million Weibo posts (China's Twitter), 2010–2017, with a protest word. Most are not about protests: "my stomach is protesting."

The method. Two neural networks give the probability that a post describes a protest: one reads the *text*, one looks at the *photos* (a network pretrained on 1.2 million everyday images, retrained on protest photos). Labels: an activists' archive of protest posts (yes), random posts (no).



Photos ordered by predicted protest probability (their Figure 3).

Zhang and Pan: Validation and Results

Honest evaluation. Checked on 10,000 posts that coders read by hand and the networks never saw. *Precision*: share of flagged posts that are really protests. *Recall*: share of real protest posts found. F1 combines the two (1 = perfect).

	precision	recall	F1
stage 1: trained on the archive vs. random posts	0.66	0.73	0.69
stage 2: retrained on 40,505 hand-coded hard cases	0.79	0.90	0.84

- Stage 2's hard cases: posts *about* grievances (demolitions, police) that describe no protest. Of the 40,505, only 9,761 were protests. Text predicted better than photos.
- **Result**: 136,330 likely protest events in over 96% of China's counties. Unpaid wages 29%, property 26%, land 17%: more rural land protests than newspaper-based data.
- **Blind spots**: few events in Tibet and Xinjiang, where social media use is lower and control tighter.

Word Embeddings

Why Turn Text and Images Into Numbers?

Sociologists increasingly study behaviour through text, images, and audio: speeches, posts, court records, street photographs (Lazer et al. 2009). A regression needs numbers. Three uses:

1. **As the object of study**: the numbers for words carry cultural meaning. How did the meaning of "class" change over a century of books? (Kozlowski, Taddy, and Evans 2019)
2. **As predictors or measurements**: score each presidential speech for populism; read a neighbourhood's income from its cars (Bonikowski, Luo, and Stuhler 2022; Gebru et al. 2017).
3. **As control variables**: hold fixed what is written in a text (next week).

This section: how word vectors are made.

A Word Is a Category

In a regression, a word is a categorical variable: one 0/1 dummy per word in the vocabulary.

	beer	wine	tesgüino	corn	wheat	drink	...
beer	1	0	0	0	0	0	...
wine	0	1	0	0	0	0	...
corn	0	0	0	1	0	0	...

This row is called a **one-hot** vector. Two problems:

- **Too long**: English needs about 100,000 columns, one per word.
- **No similarity**: beer and wine share no 1, and neither do beer and corn. The coding says beer is as different from wine as from corn.

Goal: give each word a short list of K numbers (often $K = 300$), its **word vector**, so that words with similar meanings get similar lists.

You Know a Word by Its Neighbours ★

What is tesgüino?

- A bottle of tesgüino is on the table.
- Everyone likes tesgüino.
- Tesgüino makes you drunk.
- We make tesgüino out of corn.

Jurafsky and Martin (2025), Ch. 6.

- You never saw the word, yet you know: an alcoholic drink, made from corn.
- **Distributional hypothesis**: words used in similar contexts have similar meanings.
- So describe each word by the company it keeps, then compress that description.

Word2Vec: Turn Neighbours Into a Prediction Task

A toy text of seven three-word sentences (figs/w12_lang.R, A; hypothetical):

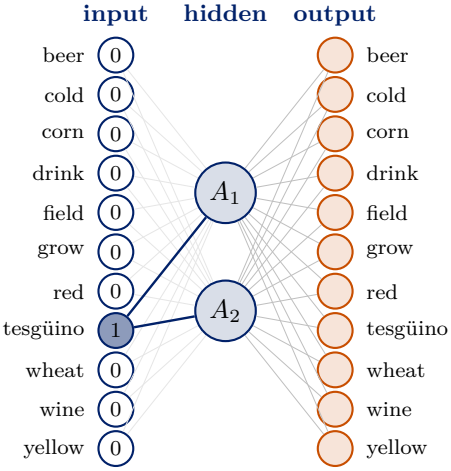
drink beer cold *drink wine cold* *drink tesgüino cold* *drink wine red*
grow corn field *grow wheat field* *grow corn yellow*

Step 1: list the pairs. For every word w, each word c right next to it is a *context* word. From *drink beer cold*:

word w	drink	beer	beer	cold
context c	beer	drink	cold	beer

- Seven sentences give 28 pairs; 11 different words. Tesgüino's pairs: (tesgüino, drink), (tesgüino, cold).
- **Step 2**: fit a model that predicts c from w. Nobody labels anything: the text is the data.

Word2Vec: the Network



The single-layer network from before, with three choices:

- **Input**: the one-hot row of word w; 11 inputs, one of them 1.
- **Hidden layer**: $K = 2$ units, $A_k = \sum_j w_{kj} X_j$, with *no activation and no intercept*: just a weighted sum.
- **Output**: 11 units, one per word, giving $\Pr(c \mid w)$ for every possible context word c.

Weights: 11×2 into the hidden layer, 2×11 out of it: 44 in all. The only nonlinear step is the softmax at the output. Word2vec is kept this simple on purpose: it must run over billions of words.

Why the Hidden Layer Is the Word's Vector

1. It is the word's own weights. Feed tesgüino's one-hot row: $X_j = 1$ for tesgüino, 0 for the other 10 words.

$$A_k = \sum_j w_{kj} X_j = w_{k,\text{beer}} \cdot 0 + \cdots + w_{k,\text{tesgüino}} \cdot 1 + \cdots = w_{k,\text{tesgüino}}$$

So the hidden layer is the pair of weights leaving tesgüino's input. Call it $u_{\text{tesgüino}} = (w_{1,\text{tesgüino}},\ w_{2,\text{tesgüino}})$: **the word vector**. Every word has its own.

2. Why it carries meaning.

- The output layer sees *only* u_w. Whatever the network predicts about w's neighbours, it must predict from these two numbers.
- Beer and tesgüino have the same neighbours (*drink*, *cold*). To predict them well, training must give both words the same u.
- Words used in similar contexts end up with similar vectors: the distributional hypothesis, turned into a prediction problem.

The Output Is Week 4's Multinomial Logit

Each possible context word c also has two output weights $v_c = (v_{c1}, v_{c2})$. Score and probability:

$$\text{score}(w, c) = \sum_k u_{wk}\, v_{ck}, \qquad \Pr(c \mid w) = \frac{\exp(\text{score}(w, c))}{\sum_{c'} \exp(\text{score}(w, c'))}$$

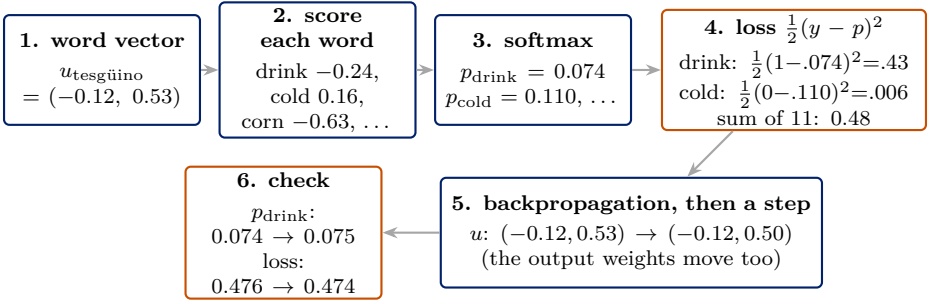
The multinomial logit with 11 categories: positive, and sums to 1 over c. Trained toy, $w =$ tesgüino, $u = (0.05, 1.35)$ (figs/w12_lang.R, A2):

context c	v_c	score	$\Pr(c \mid \text{tesgüino})$
drink	$(0.05, 1.91)$	$0.05(0.05) + 1.35(1.91) = 2.59$	0.43
cold	$(0.09, 1.68)$	$0.05(0.09) + 1.35(1.68) = 2.28$	0.31
corn	$(0.36, -2.10)$	$0.05(0.36) + 1.35(-2.10) = -2.82$	0.00

- Denominator: exp(score) summed over all 11 words = 31.09. So $\Pr(\text{drink}) = e^{2.59}/31.09 = 0.43$.

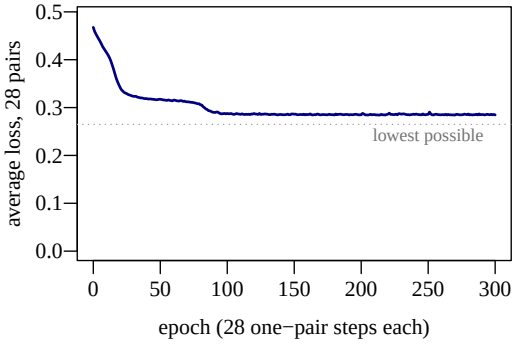
Training Word2Vec, One Pair at a Time

Stochastic gradient descent, one pair per step. Start from random weights; first pair (tesgüino, drink); learning rate 0.5 (figs/w12_lang.R, A1).



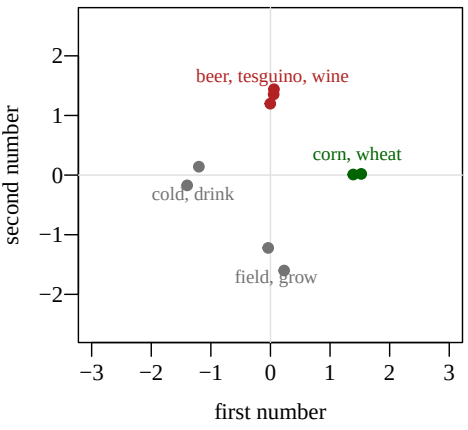
$y = 1$ for the word that appeared (drink) and 0 for the other 10. Each step moves the weights so that drink becomes a little more likely after tesgüino and the other words a little less.

Training Word2Vec: From One Pair to the Whole Text



- **Epoch**: take the 28 pairs in a random order and do the one-pair step for each: 28 small steps.
- After each epoch, average the loss over all 28 pairs (figs/w12_lang.R, A): 0.47 at the start, 0.46 after 1 epoch, 0.41 after 10, 0.29 after 300.
- It cannot reach 0. After *drink* come *beer*, *wine*, *tesgüino*, *wine*: no rule can be sure which. The lowest possible average is 0.27.
- Software averages the errors of a mini-batch (say 32 pairs), then steps. All 28 at once is plain gradient descent.

Word2Vec by Hand: the Result



The 11 trained word vectors u_w (figs/w12_lang.R, A). Similarity is measured by the **cosine** of two vectors: 1 = same direction, 0 = unrelated, −1 = opposite.

- Beer, tesgüino, wine sit together: each must predict *drink* and *cold*. Cosine of tesgüino with beer 1.00, with wine 1.00.
- Corn and wheat sit together elsewhere (both predict *grow*, *field*): tesgüino and corn, 0.05, unrelated.
- We never told the network that tesgüino is a drink. It learned it from the company the word keeps.

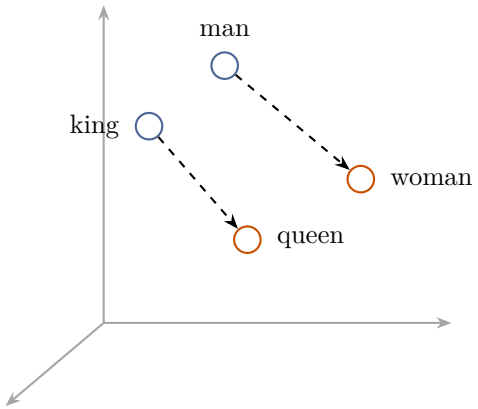
Negative Sampling: the Shortcut Used in Practice ★

The softmax needs a score for *every* word in the vocabulary, about 100,000 of them, at every step, for billions of pairs. Too slow. Instead, at each step:

1. **Pull together**: take the real pair, say (tesgüino, drink), and move tesgüino's vector toward drink's output vector.
2. **Push apart**: draw a few words at random from the vocabulary, say *carburetor* and *field*, treat them as fake neighbours, and move tesgüino's vector away from theirs.
3. Leave the other 99,997 words alone.

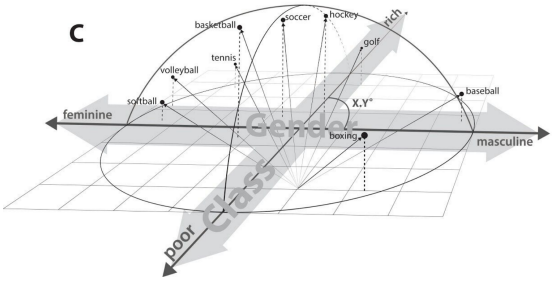
- Each step becomes a yes/no question, "real neighbour or random word?", so the work per step does not grow with the vocabulary.
- The number of random words per real pair is a tuning parameter: 5 by default in word2vec (`negative = 5` in R's `word2vec`); 5–20 for small texts (Mikolov et al. 2013).

Directions Carry Meaning



- The step from *man* to *woman* is roughly the same arrow as from *king* to *queen*: one direction carries "male to female."
- So vector(*king*) − vector(*man*) + vector(*woman*) lands closest to vector(*queen*) (Mikolov et al. 2013).
- In Lab 12, on Jane Austen's novels: *brother* − *he* + *she* lands on *sister*.
- Sociology: average several such arrows into one *dimension* (gender, class), then place any word on it. Next slides.

In the Literature: the Geometry of Culture



The question. Kozlowski, Taddy, and Evans (2019): what does "class" mean, and how did its meaning change over the twentieth century?

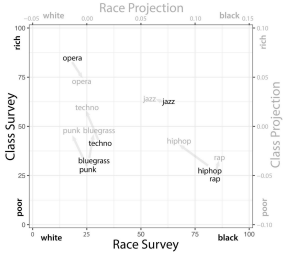
The method. Word vectors trained on books and news.

1. **Build a dimension**: average arrows over many antonym pairs, *rich* − *poor*, *affluent* − *impoverished*, . . . : a class axis. The same for gender.
2. **Place words on it**: a word's cosine with each axis.

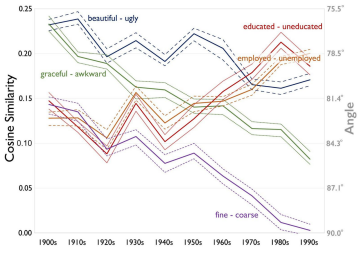
Their Figure 2C: sports on gender and class at once. Golf: rich, masculine. Boxing: poor, masculine. Softball, volleyball: feminine.

Kozlowski et al.: Validation and Results

Class axis: the average of 8 arrows, *rich* − *poor*, *affluent* − *impoverished*, *expensive* − *inexpensive*, *luxury* − *cheap*, … (their Table 1). A word's class score is its cosine with this axis.



Do the axes match people? Music genres: embedding (gray) vs. survey ratings (black). Over 59 words they correlate 0.88 for gender, 0.70 for race, 0.52 for class (their Table 2).



How class changed. Each dimension's cosine with the class axis, by decade, U.S. books. In 1900 class meant *beautiful*, *graceful*, *fine*; by 1990, *educated* and *employed*.

Interpretation

Saying It Right: Prediction, Not Cause ★

"Is this variable important for predicting civil war (or a disease)?" Yes, in that form: important *for prediction*, *in this model*, *in these data*. GSS rows, then two hypothetical:

do not say	**say instead**
Children do not affect earnings (the lasso dropped it).	With the other 22 columns in, children add little to prediction; the lasso kept it in 9% of bootstrap samples.
Education is the most important cause of earnings.	The forest's test MSE rises most when education is shuffled (0.645 → 0.809): its predictions lean on education most.
Poverty causes civil war (it tops the importance list).	Poverty is a strong predictor of onset in these country-years; whether it causes war is a design question (Weeks 5–11).
Gene X is *the* driver of the disease.	X improves prediction of the disease; a correlated gene could have taken its place.

Why: selection and importance follow correlation with Y; correlated variables can swap places; another sample can give another list.

Looking Ahead

- Every method today trades bias for variance, tuned by cross-validation.
- That trade is exactly why we cannot drop these estimates into a regression and read off a coefficient: they are *deliberately biased* and converge slowly.
- **Next week**: principal components, text as a control variable, and what has to change before an estimate this biased can be used as an ingredient in a causal estimator.

Reading

ISL Chapters 5, 6, and 8; Molina and Garip (2019). Optional: ISL Chapter 10 (deep learning); Kleinberg et al. (2015) on prediction policy problems.

References

- Athey, Susan and Guido W. Imbens. 2019. "Machine Learning Methods That Economists Should Know About." *Annual Review of Economics* 11: 685–725.
- Baldi, Pierre and Kurt Hornik. 1989. "Neural Networks and Principal Component Analysis: Learning from Examples without Local Minima." *Neural Networks* 2(1): 53–58.
- Blumenstock, Joshua, Gabriel Cadamuro, and Robert On. 2015. "Predicting Poverty and Wealth from Mobile Phone Metadata." *Science* 350(6264): 1073–1076.
- Bonikowski, Bart, Yuchen Luo, and Oscar Stuhler. 2022. "Politics as Usual? Measuring Populism, Nationalism, and Authoritarianism in U.S. Presidential Campaigns (1952–2020) with Neural Language Models." *Sociological Methods & Research* 51(4): 1721–1787.
- Breiman, Leo. 2001. "Random Forests." *Machine Learning* 45(1): 5–32.
- Chen, Tianqi and Carlos Guestrin. 2016. "XGBoost: A Scalable Tree Boosting System." *Proceedings of KDD '16*: 785–794.

References (cont.)

- Chernozhukov, Victor, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, Whitney Newey, and James Robins. 2018. "Double/Debiased Machine Learning for Treatment and Structural Parameters." *The Econometrics Journal* 21(1): C1–C68.
- Gebru, Timnit, et al. 2017. "Using Deep Learning and Google Street View to Estimate the Demographic Makeup of Neighborhoods across the United States." *PNAS* 114(50): 13108–13113.
- Grinsztajn, Léo, Edouard Oyallon, and Gaël Varoquaux. 2022. "Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?" *NeurIPS Datasets and Benchmarks*.
- Hastie, Trevor, Robert Tibshirani, and Jerome Friedman. 2009. *The Elements of Statistical Learning*, 2nd ed. Springer.
- Hegre, Håvard and Nicholas Sambanis. 2006. "Sensitivity Analysis of Empirical Results on Civil War Onset." *Journal of Conflict Resolution* 50(4): 508–535.
- Hornik, Kurt, Maxwell Stinchcombe, and Halbert White. 1989. "Multilayer Feedforward Networks Are Universal Approximators." *Neural Networks* 2(5): 359–366.

References (cont.)

- Imai, Kosuke and Kentaro Nakamura. 2026. "Leveraging Generative AI for Causal Inference with Unstructured Data." *Proceedings of the National Academy of Sciences* 123(36): e2530532123.
- James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2021. *An Introduction to Statistical Learning*, 2nd ed. Springer. [Ch. 5, 6, 8, 10]
- Joel, Samantha, Paul W. Eastwick, Colleen J. Allison, et al. 2020. "Machine Learning Uncovers the Most Robust Self-Report Predictors of Relationship Quality across 43 Longitudinal Couples Studies." *PNAS* 117(32): 19061–19071.
- Jurafsky, Daniel and James H. Martin. 2025. *Speech and Language Processing*, 3rd ed. draft. Ch. 6, "Vector Semantics and Embeddings."
- Kleinberg, Jon, Himabindu Lakkaraju, Jure Leskovec, Jens Ludwig, and Sendhil Mullainathan. 2018. "Human Decisions and Machine Predictions." *Quarterly Journal of Economics* 133(1): 237–293.
- Kleinberg, Jon, Jens Ludwig, Sendhil Mullainathan, and Ziad Obermeyer. 2015. "Prediction Policy Problems." *American Economic Review: Papers & Proceedings* 105(5): 491–495.

References (cont.)

- Kozlowski, Austin C., Matt Taddy, and James A. Evans. 2019. "The Geometry of Culture: Analyzing the Meanings of Class through Word Embeddings." *American Sociological Review* 84(5): 905–949.
- Lazer, David, Alex Pentland, Lada Adamic, Sinan Aral, Albert-László Barabási, Devon Brewer, et al. 2009. "Computational Social Science." *Science* 323(5915): 721–723.
- Mikolov, Tomas, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. "Distributed Representations of Words and Phrases and Their Compositionality." *Advances in Neural Information Processing Systems* 26.
- Molina, Mario and Filiz Garip. 2019. "Machine Learning for Sociology." *Annual Review of Sociology* 45: 27–45.
- Muchlinski, David, David Siroky, Jingrui He, and Matthew Kocher. 2016. "Comparing Random Forest with Logistic Regression for Predicting Class-Imbalanced Civil War Onset Data." *Political Analysis* 24(1): 87–103.
- Mullainathan, Sendhil and Jann Spiess. 2017. "Machine Learning: An Applied Econometric Approach." *Journal of Economic Perspectives* 31(2): 87–106.

References (cont.)

- Salganik, Matthew J., Ian Lundberg, Alexander T. Kindel, et al. 2020. "Measuring the Predictability of Life Outcomes with a Scientific Mass Collaboration." *PNAS* 117(15): 8398–8403.
- Strobl, Carolin, Anne-Laure Boulesteix, Achim Zeileis, and Torsten Hothorn. 2007. "Bias in Random Forest Variable Importance Measures." *BMC Bioinformatics* 8: 25.
- Tibshirani, Robert. 1996. "Regression Shrinkage and Selection via the Lasso." *Journal of the Royal Statistical Society Series B* 58(1): 267–288.
- van der Laan, Mark J., Eric C. Polley, and Alan E. Hubbard. 2007. "Super Learner." *Statistical Applications in Genetics and Molecular Biology* 6(1): 25.
- Verhagen, Mark D. 2024. "Incorporating Machine Learning into Sociological Model-Building." *Sociological Methodology* 54(2): 217–268.
- Wager, Stefan and Susan Athey. 2018. "Estimation and Inference of Heterogeneous Treatment Effects Using Random Forests." *Journal of the American Statistical Association* 113(523): 1228–1242.

References (cont.)

- Zou, Hui and Trevor Hastie. 2005. "Regularization and Variable Selection via the Elastic Net." *Journal of the Royal Statistical Society Series B* 67(2): 301–320.
- Zhang, Han and Jennifer Pan. 2019. "CASM: A Deep-Learning Approach for Identifying Collective Action Events with Text and Image Data from Social Media." *Sociological Methodology* 49(1): 1–57.